wojtekmaj / **react-pdf**

Display PDFs in your React app as easily as if they were images.

projects.wojtekmaj.pl/react-pdf

MIT License

3.9k stars **397** forks

Star Watch

Code Issues 107 Pull requests 6 Actions Projects Wiki Security ···

master ···

wojtekmaj ··· 4 days ago

View code

README.md

npm v5.0.0 downloads 11M CI passing dependencies up to date dev dependencies up to date tested with jest

React-PDF

Display PDFs in your React app as easily as if they were images.

tl;dr

- Install by executing `npm install react-pdf` or `yarn add react-pdf`.
- Import by adding `import { Document } from 'react-pdf'`.
- Use by adding `<Document file="..." />`. `file` can be a URL, base64 content, Uint8Array, and more.
- Put `<Page />` components inside `<Document />` to render pages.

Demo

A minimal demo page can be found in `sample` directory.

Online demo is also available!

Before you continue

React-PDF is under constant development. This documentation is written for React-PDF 5.x branch. If you want to see documentation for other versions of React-PDF, use dropdown on top of GitHub page to switch to an appropriate tag. Here are quick links to the newest docs from each branch:

- v4.x
- v3.x
- v2.x
- v1.x

Getting started

Compatibility

React

To use the latest version of React-PDF, your project needs to use React 16.3 or later.

If you use an older version of React, please refer to the table below to a find suitable React-PDF version. Don't worry - as long as you're running React 15.5 or later, you won't be missing out a lot!

React version	Newest compatible React-PDF version
≥16.3	latest
≥15.5	4.x

Internet Explorer

Internet Explorer is not supported in React-PDF 5.x or later. If you need Internet Explorer support, you will need to use React-PDF 4.x instead. Don't worry - it still receives bug fixes and even occasional small features!

Installation

Add React-PDF to your project by executing `npm install react-pdf` or `yarn add react-pdf`.

Usage

Here's an example of basic usage:

```
import React, { useState } from 'react';
import { Document, Page } from 'react-pdf';

function MyApp() {
  const [numPages, setNumPages] = useState(null);
  const [pageNumber, setPageNumber] = useState(1);

  function onDocumentLoadSuccess({ numPages }) {
    setNumPages(numPages);
  }

  return (
    <div>
      <Document
        file="somefile.pdf"
        onLoadSuccess={onDocumentLoadSuccess}
      >
        <Page pageNumber={pageNumber} />
      </Document>
      <p>Page {pageNumber} of {numPages}</p>
    </div>
  );
}
```

Check the sample directory in this repository for a full working example. For more examples and more advanced use cases, check Recipes in React-PDF Wiki.

Enable PDF.js worker

It is crucial for performance to use PDF.js worker whenever possible. This ensures that PDF files will be rendered in a separate thread without affecting page performance. To make things a little easier, we've prepared several entry points you can use.

Webpack

Instead of directly importing/requiring `'react-pdf'`, import it like so:

```
// using ES6 modules
import { Document } from 'react-pdf/dist/esm/entry.webpack';

// using CommonJS modules
import { Document } from 'react-pdf/dist/umd/entry.webpack';
```

Parcel

Instead of directly importing/requiring `'react-pdf'`, import it like so:

```
// using ES6 modules
import { Document } from 'react-pdf/dist/esm/entry.parcel';

// using CommonJS modules
import { Document } from 'react-pdf/dist/umd/entry.parcel';
```

Create React App

Create React App uses Webpack under the hood, but instructions for Webpack will not work. Standard instructions apply.

Standard (Browserify and others)

If you use Browserify or other bundling tools, you will have to make sure on your own that `pdf.worker.js` file from `pdfjs-dist/build` is copied to your project's output folder.

Alternatively, you could use `pdf.worker.js` from an external CDN:

```
import { pdfjs } from 'react-pdf';
pdfjs.GlobalWorkerOptions.workerSrc = `//cdnjs.cloudflare.com/ajax/libs/pdf.js
```

Support for annotations

If you want to use annotations (e.g. links) in PDFs rendered by React-PDF, then you would need to include stylesheet necessary for annotations to be correctly displayed like so:

```
// using ES6 modules
import 'react-pdf/dist/esm/Page/AnnotationLayer.css';

// using CommonJS modules
import 'react-pdf/dist/umd/Page/AnnotationLayer.css';
```

Support for non-latin characters

If you want to ensure that PDFs with non-latin characters will render perfectly, or you have encountered the following warning:

```
Warning: The CMap "baseUrl" parameter must be specified, ensure that the
"cMapUrl" and "cMapPacked" API parameters are provided.
```

then you would also need to include cMaps in your build and tell React-PDF where they are.

Copying cMaps

First, you need to copy cMaps from `pdfjs-dist` (React-PDF's dependency - it should be in your `node_modules` if you have React-PDF installed). cMaps are located in `pdfjs-dist/cmaps`.

Webpack

Add `copy-webpack-plugin` to your project if you haven't already:

```
npm install copy-webpack-plugin --save-dev
```

Now, in your Webpack config, import the plugin:

```
import CopyWebpackPlugin from 'copy-webpack-plugin';
```

and in `plugins` section of your config, add the following:

```
new CopyWebpackPlugin([
  {
    from: 'node_modules/pdfjs-dist/cmaps/',
    to: 'cmaps/'
  },
]),
```

Parcel, Browserify and others

If you use Parcel, Browserify or other bundling tools, you will have to make sure on your own that cMaps are copied to your project's output folder.

Setting up React-PDF

Now that you have cMaps in your build, pass required options to Document component by using `options` prop, like so:

```
<Document
  options={{
    cMapUrl: 'cmaps/',
    cMapPacked: true,
  }}
/>
```

Alternatively, you could use cMaps from external CDN:

```
import { pdfjs } from 'react-pdf';

<Document
  options={{
    cMapUrl: `//cdn.jsdelivr.net/npm/pdfjs-dist@${pdfjs.version}/cmaps/`,
    cMapPacked: true,
  }}
/>
```

User guide

Document

Loads a document passed using `file` prop.

Props

Prop name	Description	Default value	Exampl
className	Class name(s) that will be added to rendered element along with the default `react-pdf__Document`.	n/a	• String: `"custom-class class-name-2"` • Array of strings: `["custom-clas "custom-class-`
error	What the component should display in case of an error.	`"Failed to load PDF file."`	• String: `"An error occ` • React element: `<div>An error` • Function: `this.renderEr`
externalLinkTarget	Link target for external links rendered in annotations.	unset, which means that default behavior will be used	One of valid values f • `"_self"` • `"_blank"` • `"_parent"` • `"_top"`

Prop name	Description	Default value	Exampl
file	What PDF should be displayed. Its value can be an URL, a file (imported using `import ... from ...` or from file input form element), or an object with parameters (`url` - URL; `data` - data, preferably Uint8Array; `range` - PDFDataRangeTransport; `httpHeaders` - custom request headers, e.g. for authorization), `withCredentials` - a boolean to indicate whether or not to include cookies in the request (defaults to `false`). **Warning**: Since equality check (`===`) is used to determine if `file` object has changed, it must be memoized by setting it in component's state, `useMemo` or other similar technique.	n/a	• URL: `"http://examp` • File: `import sample` `'../static/sam` `sample` • Parameter obje `{ url:` `'http://exampl` `httpHeaders: {` `'4035982095802` `withCredential`

Prop name	Description	Default value	Exampl
inputRef	A prop that behaves like ref, but it's passed to main `<div>` rendered by `<Document>` component.	n/a	• Function: `(ref) => { th ref; }` • Ref created usi `React.createR` `this.ref = Re` … `inputRef={thi` • Ref created usi `const ref = R` … `inputRef={ref`
loading	What the component should display while loading.	`"Loading PDF…"`	• String: `"Please wait!` • React element: `<div>Please w` • Function: `this.renderLo`
noData	What the component should display in case of no data.	`"No PDF file specified."`	• String: `"Please selec` • React element: `<div>Please s` `</div>` • Function: `this.renderNo`
onItemClick	Function called when an outline item has been clicked. Usually, you would like to use this callback to move the user wherever they requested to.	n/a	`({ pageNumber })` `an item from page` `'!')`
onLoadError	Function called in case of an error while loading a document.	n/a	`(error) => alert(` `loading document!` `error.message)`

Prop name	Description	Default value	Exampl
onLoadSuccess	Function called when the document is successfully loaded.	n/a	`(pdf) => alert('L ' + pdf.numPages +`
onPassword	Function called when a password-protected PDF is loaded.	A function that prompts the user for password	`(callback) => callback('s3cr3t_p`
onSourceError	Function called in case of an error while retrieving document source from `file` prop.	n/a	`(error) => alert( retrieving documer error.message)`
onSourceSuccess	Function called when document source is successfully retrieved from `file` prop.	n/a	`() => alert('Docu retrieved!')`
options	An object in which additional parameters to be passed to PDF.js can be defined. For a full list of possible parameters, check PDF.js documentation on DocumentInitParameters.	n/a	`{ cMapUrl: 'cmaps true }`
renderMode	Rendering mode of the document. Can be `"canvas"`, `"svg"` or `"none"`.	`"canvas"`	`"svg"`
rotate	Rotation of the document in degrees. If provided, will change rotation globally, even for the pages which were given `rotate` prop of their own. `90` = rotated to the right, `180` = upside down, `270` = rotated to the left.	n/a	`90`

Page

Displays a page. Should be placed inside `<Document />`. Alternatively, it can have `pdf` prop passed, which can be obtained from `<Document />`'s `onLoadSuccess` callback function, however some advanced functions like linking between pages inside a document may not be working correctly.

Props

Prop name	Description	Default value	Exampl
className	Class name(s) that will be added to rendered element along with the default `react-pdf__Page`.	n/a	• String: `"custo name-1 class-n` • Array of `["custo name-1" class-n`
customTextRenderer	A function that customizes how a text layer is rendered. Passes itext item and index for item.	n/a	`({ str, it => { return {str}</mark`
error	What the component should display in case of an error.	`"Failed to load the page."`	• String: `"An er occurre` • React e `<div>A occurre` • Functio `this.r`

Prop name	Description	Default value	Exampl
height	Page height. If neither `height` nor `width` are defined, page will be rendered at the size defined in PDF. If you define `width` and `height` at the same time, `height` will be ignored. If you define `height` and `scale` at the same time, the height will be multiplied by a given factor.	Page's default height	`300`
inputRef	A prop that behaves like ref, but it's passed to main `<div>` rendered by `<Page>` component.	n/a	• Functior `(ref) =` `this.my` `}` • Ref crea `React.` `this.r` `React.c` ... `inputR` `{this.r` • Ref crea `React.` `const` `React.u` ... `inputR`
loading	What the component should display while loading.	`"Loading page…"`	• String: `"Pleas` • React e `<div>P` `</div>` • Functior `this.r`

Prop name	Description	Default value	Exampl
noData	What the component should display in case of no data.	`"No page specified."`	• String: `"Pleas` `page."` • React e `<div>P` `a page.` • Functior `this.r`
onLoadError	Function called in case of an error while loading the page.	n/a	`(error) =>` `alert('Erro` `loading pag` `error.messa`
onLoadProgress	Function called, potentially multiple times, as the loading progresses.	n/a	`({ loaded,` `alert('Load` `document: '` `/ total) *`
onLoadSuccess	Function called when the page is successfully loaded.	n/a	`(page) =>` `displaying` `number ' +` `page.pageNu`
onRenderError	Function called in case of an error while rendering the page.	n/a	`(error) =>` `alert('Erro` `loading pag` `error.messa`
onRenderSuccess	Function called when the page is successfully rendered on the screen.	n/a	`() => aler` `the page!')`
onGetAnnotationsSuccess	Function called when annotations are successfully loaded.	n/a	`(annotatio` `alert('Now` `' + annotat` `+ ' annotat`
onGetAnnotationsError	Function called in case of an error while loading annotations.	n/a	`(error) =>` `alert('Erro` `loading ann` `+ error.mes`

Prop name	Description	Default value	Exampl
onGetTextSuccess	Function called when text layer items are successfully loaded.	n/a	`(items) => displaying items.lengt layer items`
onGetTextError	Function called in case of an error while loading text layer items.	n/a	`(error) => alert('Erro loading tex items! ' + error.messa`
pageIndex	Which page from PDF file should be displayed, by page index.	`0`	`1`
pageNumber	Which page from PDF file should be displayed, by page number. If provided, `pageIndex` prop will be ignored.	`1`	`2`
renderAnnotationLayer	Whether annotations (e.g. links) should be rendered.	`true`	`false`
renderInteractiveForms	Whether interactive forms should be rendered. `renderAnnotationLayer` prop must be set to `true`.	`false`	`true`
renderMode	Rendering mode of the document. Can be `"canvas"`, `"svg"` or `"none"`.	`"canvas"`	`"svg"`
renderTextLayer	Whether a text layer should be rendered.	`true`	`false`
rotate	Rotation of the page in degrees. `90` = rotated to the right, `180` = upside down, `270` = rotated to the left.	Page's default setting, usually `0`	`90`

Prop name	Description	Default value	Exampl
scale	Page scale.	`1.0`	`0.5`
width	Page width. If neither `height` nor `width` are defined, page will be rendered at the size defined in PDF. If you define `width` and `height` at the same time, `height` will be ignored. If you define `width` and `scale` at the same time, the width will be multiplied by a given factor.	Page's default width	`300`

Outline

Displays an outline (table of contents). Should be placed inside `<Document />`. Alternatively, it can have `pdf` prop passed, which can be obtained from `<Document />`'s `onLoadSuccess` callback function.

Props

Prop name	Description	Default value	Example values
className	Class name(s) that will be added to rendered element along with the default `react-pdf__Outline`.	n/a	• String: `"custom-class-name-1 custom-class-name-2"` • Array of strings: `["custom-class-name-1", "custom-class-name-2"]`

Prop name	Description	Default value	Example values
inputRef	A prop that behaves like ref, but it's passed to main `<div>` rendered by `<Outline>` component.	n/a	• Function: `(ref) => { this.myOutline = ref; }` • Ref created using `React.createRef`: `this.ref = React.createRef();` … `inputRef={this.ref}` • Ref created using `React.useRef`: `const ref = React.useRef();` … `inputRef={ref}`
onItemClick	Function called when an outline item has been clicked. Usually, you would like to use this callback to move the user wherever they requested to.	n/a	`({ pageNumber }) => alert('Clicked an item from page ' + pageNumber + '!')`
onLoadError	Function called in case of an error while retrieving the outline.	n/a	`(error) => alert('Error while retrieving the outline! ' + error.message)`
onLoadSuccess	Function called when the outline is successfully retrieved.	n/a	`(outline) => alert('The outline has been successfully retrieved.')`

Useful links

- React-PDF Wiki

License

The MIT License.

Author

	Wojciech Maj kontakt@wojtekmaj.pl http://wojtekmaj.pl

Thank you

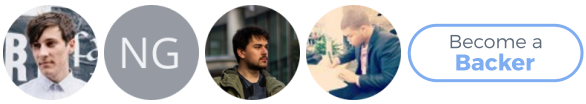
This project wouldn't be possible without awesome work of Niklas Närhinen niklas@narhinen.net who created its initial version and without Mozilla, author of pdf.js. Thank you!

Sponsors

Thank you to all our sponsors! Become a sponsor and get your image on our README on GitHub.

Backers

Thank you to all our backers! Become a backer and get your image on our README on GitHub.



Top Contributors

Thank you to all our contributors that helped on this project!

Releases 84

v5.0.0 Latest
on Aug 27

+ 83 releases

Sponsor this project

wojtekmaj Wojciech Maj

opencollective.com/**react-pdf-wojtekmaj**

Learn more about GitHub Sponsors

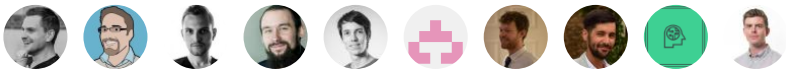
Packages

No packages published

Used by 7.1k

+ 7,128

Contributors 38



+ 27 contributors

Languages

- **JavaScript** 94.4%
- **CSS** 3.5%
- **Less** 1.9%
- **HTML** 0.2%